AIDS VIRUS INFECTION PREDICTION

EZO YEDİGÖL – ÜMMÜ SAVRAN

2024-06-09

SUBJECT

- AIDS (Acquired Immunodeficiency Syndrome) is a disease caused by the HIV (Human Immunodeficiency Virus). It weakens the immune system, making it hard for the body to fight off infections and certain cancers.
- Diagnosing AIDS usually takes a few days to a few weeks, which can be a disadvantage for patients. Also, there is a higher risk of spreading the virus during this time due to patient negligence.
- If the diagnosis process is faster, it benefits patients and saves time and money in healthcare.
- Using machine learning methods can speed up the diagnosis, allowing patients to get treatment sooner and making the healthcare system more efficient.

DATA SET: EDA

- Dataset contains healthcare statistics and categorical information about patients who have been diagnosed with AIDS. This dataset was initially published in 1996.
- The dataset contains 2139 observations and 23 variables. The target variable is "infected", and the remaining 22 variables are features.
- FEATURES: drugs: history of IV drug use (0=no, 1=yes) hemo: hemophilia (0=no, 1=yes) homo: homosexual activity (0=no, 1=yes) gender: gender (0=F, 1=M) offtrt: indicator of off-trt before 96+/-5 weeks (0=no,1=yes) oprior: Non-ZDV antiretroviral therapy pre-175 (0=no, 1=yes) race: race (0=White, 1=non-white) str2: antiretroviral history (0=naive, 1=experienced) z30: ZDV in the 30 days prior to 175 (0=no, 1=yes) strat: antiretroviral history stratification (1='Antiretroviral Naive',2='> 1 but <= 52 weeks of prior antiretroviral therapy',3='> 52 weeks) symptom: symptomatic indicator (0=asymp, 1=symp) treat: treatment indicator (0=ZDV only, 1=others) trt: treatment indicator (0 = ZDV only; 1 = ZDV + ddI, 2 = ZDV + Zal, 3 = ddI only) str2: antiretroviral history (0=naive, 1=experienced) time: time to failure or censoring age: age (yrs) at baseline wtkg: weight (kg) at baseline karnof: Karnofsky score (on a scale of 0-100) preanti: days pre-175 anti-retroviral therapy cd40: CD4 at baseline cd420: CD4 at 20+/-5 weeks cd80: CD8 at baseline cd820: CD8 at 20+/-5 weeks

```r
# LOADİNG DATASET

library(readr)
AIDS_Classification <-
read_csv("C:/Users/Lenovo/Downloads/AIDS_Classification.csv")
new_data<- AIDS_Classification
```

- Information about the dataset

```r
# Display the structure of the dataset
str(new_data)
```

```
## spc_tbl_ [2,139 × 23] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
##  $ time    : num [1:2139] 948 1002 961 1166 1090 ...
##  $ trt     : num [1:2139] 2 3 3 3 0 1 0 0 3 0 ...
##  $ age     : num [1:2139] 48 61 45 47 43 46 31 41 40 35 ...
##  $ wtkg    : num [1:2139] 89.8 49.4 88.5 85.3 66.7 ...
##  $ hemo    : num [1:2139] 0 0 0 0 0 0 0 0 0 0 ...
##  $ homo    : num [1:2139] 0 0 1 1 1 1 1 1 1 1 ...
##  $ drugs   : num [1:2139] 0 0 1 0 0 1 0 1 0 0 ...
##  $ karnof  : num [1:2139] 100 90 90 100 100 100 100 100 90 100 ...
##  $ oprior  : num [1:2139] 0 0 0 0 0 0 0 0 0 0 ...
##  $ z30     : num [1:2139] 0 1 1 1 1 1 1 1 1 1 ...
##  $ preanti : num [1:2139] 0 895 707 1399 1352 ...
##  $ race    : num [1:2139] 0 0 0 0 0 0 0 0 0 0 ...
##  $ gender  : num [1:2139] 0 0 1 1 1 1 1 1 1 1 ...
##  $ str2    : num [1:2139] 0 1 1 1 1 1 1 1 1 1 ...
##  $ strat   : num [1:2139] 1 3 3 3 3 3 3 3 3 3 ...
##  $ symptom : num [1:2139] 0 0 0 0 0 0 0 0 1 0 ...
##  $ treat   : num [1:2139] 1 1 1 1 0 1 0 0 1 0 ...
##  $ offtrt  : num [1:2139] 0 0 1 0 0 0 0 0 1 1 ...
##  $ cd40    : num [1:2139] 422 162 326 287 504 235 244 401 214 221 ...
##  $ cd420   : num [1:2139] 477 218 274 394 353 339 225 366 107 132 ...
##  $ cd80    : num [1:2139] 566 392 2063 1590 870 ...
##  $ cd820   : num [1:2139] 324 564 1893 966 782 ...
##  $ infected: num [1:2139] 0 1 0 0 0 0 1 0 1 1 ...
##  - attr(*, "spec")=
##   .. cols(
##   ..   time = col_double(),
##   ..   trt = col_double(),
##   ..   age = col_double(),
##   ..   wtkg = col_double(),
##   ..   hemo = col_double(),
##   ..   homo = col_double(),
##   ..   drugs = col_double(),
##   ..   karnof = col_double(),
##   ..   oprior = col_double(),
##   ..   z30 = col_double(),
##   ..   preanti = col_double(),
##   ..   race = col_double(),
##   ..   gender = col_double(),
```

```
##   ..   str2 = col_double(),
##   ..   strat = col_double(),
##   ..   symptom = col_double(),
##   ..   treat = col_double(),
##   ..   offtrt = col_double(),
##   ..   cd40 = col_double(),
##   ..   cd420 = col_double(),
##   ..   cd80 = col_double(),
##   ..   cd820 = col_double(),
##   ..   infected = col_double()
##   .. )
##  - attr(*, "problems")=<externalptr>
```

```r
# Calculate the total number of missing values in the dataset
sum(is.na(new_data))
```

```
## [1] 0
```

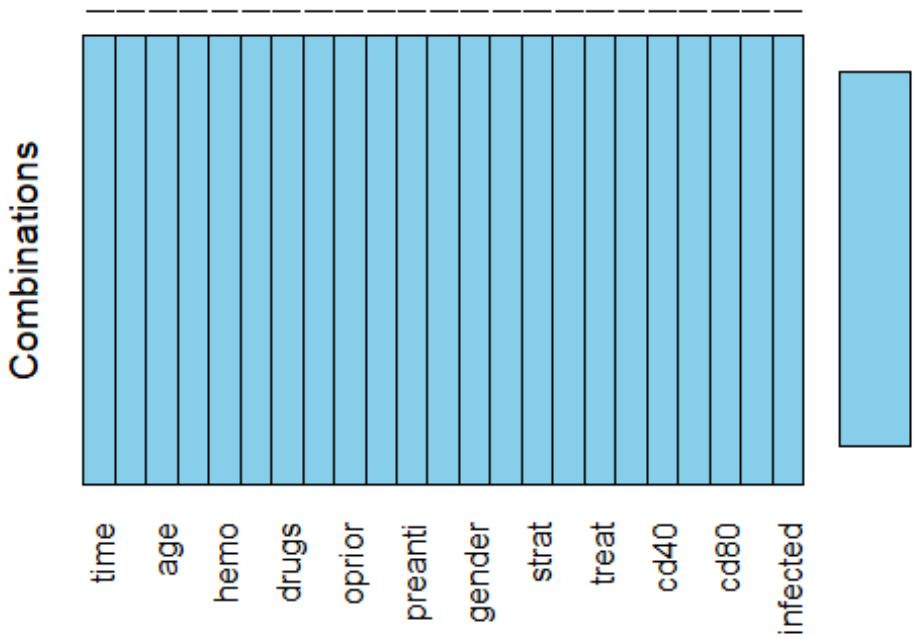
```r
# Summary statistics of the dataset
summary(new_data)
```

```
##       time             trt             age             wtkg
##  Min.   :  14.0   Min.   :0.000   Min.   :12.00   Min.   : 31.00
##  1st Qu.: 727.0   1st Qu.:1.000   1st Qu.:29.00   1st Qu.: 66.68
##  Median : 997.0   Median :2.000   Median :34.00   Median : 74.39
##  Mean   : 879.1   Mean   :1.521   Mean   :35.25   Mean   : 75.13
##  3rd Qu.:1091.0   3rd Qu.:3.000   3rd Qu.:40.00   3rd Qu.: 82.56
##  Max.   :1231.0   Max.   :3.000   Max.   :70.00   Max.   :159.94
##       hemo              homo            drugs            karnof
##  Min.   :0.00000   Min.   :0.0000   Min.   :0.0000   Min.   : 70.00
##  1st Qu.:0.00000   1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.: 90.00
##  Median :0.00000   Median :1.0000   Median :0.0000   Median :100.00
##  Mean   :0.08415   Mean   :0.6611   Mean   :0.1314   Mean   : 95.45
##  3rd Qu.:0.00000   3rd Qu.:1.0000   3rd Qu.:0.0000   3rd Qu.:100.00
##  Max.   :1.00000   Max.   :1.0000   Max.   :1.0000   Max.   :100.00
##      oprior             z30            preanti            race
##  Min.   :0.00000   Min.   :0.0000   Min.   :   0.0   Min.   :0.0000
##  1st Qu.:0.00000   1st Qu.:0.0000   1st Qu.:   0.0   1st Qu.:0.0000
##  Median :0.00000   Median :1.0000   Median : 142.0   Median :0.0000
##  Mean   :0.02197   Mean   :0.5503   Mean   : 379.2   Mean   :0.2885
##  3rd Qu.:0.00000   3rd Qu.:1.0000   3rd Qu.: 739.5   3rd Qu.:1.0000
##  Max.   :1.00000   Max.   :1.0000   Max.   :2851.0   Max.   :1.0000
##      gender           str2            strat         symptom
##  Min.   :0.000   Min.   :0.0000   Min.   :1.00   Min.   :0.000
##  1st Qu.:1.000   1st Qu.:0.0000   1st Qu.:1.00   1st Qu.:0.000
##  Median :1.000   Median :1.0000   Median :2.00   Median :0.000
##  Mean   :0.828   Mean   :0.5858   Mean   :1.98   Mean   :0.173
##  3rd Qu.:1.000   3rd Qu.:1.0000   3rd Qu.:3.00   3rd Qu.:0.000
##  Max.   :1.000   Max.   :1.0000   Max.   :3.00   Max.   :1.000
##      treat            offtrt            cd40            cd420
##  Min.   :0.0000   Min.   :0.0000   Min.   :   0.0   Min.   :  49.0
```

```
##  1st Qu.:1.0000   1st Qu.:0.0000   1st Qu.: 263.5   1st Qu.: 269.0
##  Median :1.0000   Median :0.0000   Median : 340.0   Median : 353.0
##  Mean   :0.7513   Mean   :0.3628   Mean   : 350.5   Mean   : 371.3
##  3rd Qu.:1.0000   3rd Qu.:1.0000   3rd Qu.: 423.0   3rd Qu.: 460.0
##  Max.   :1.0000   Max.   :1.0000   Max.   :1199.0   Max.   :1119.0
##       cd80            cd820           infected
##  Min.   :  40.0   Min.   : 124.0   Min.   :0.0000
##  1st Qu.: 654.0   1st Qu.: 631.5   1st Qu.:0.0000
##  Median : 893.0   Median : 865.0   Median :0.0000
##  Mean   : 986.6   Mean   : 935.4   Mean   :0.2436
##  3rd Qu.:1207.0   3rd Qu.:1146.5   3rd Qu.:0.0000
##  Max.   :5011.0   Max.   :6035.0   Max.   :1.0000
```

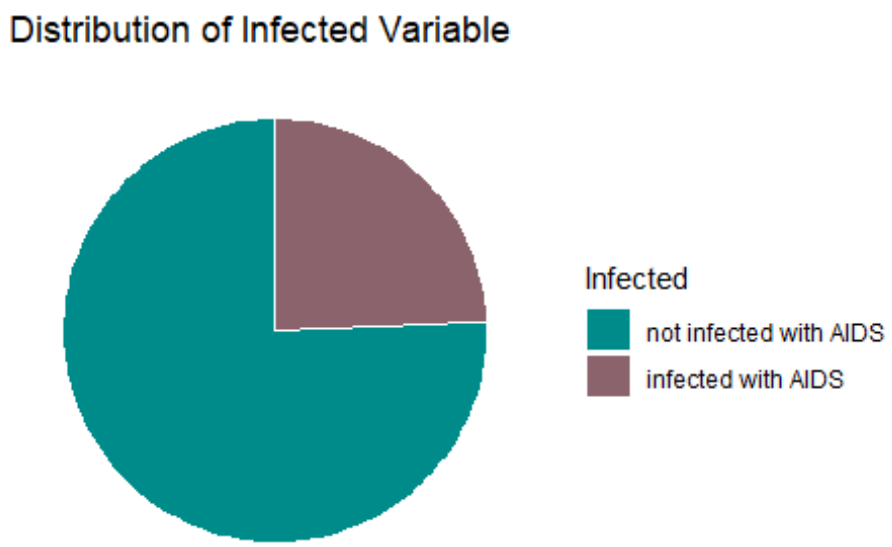
_ Observed missing observations in the dataset:

```
library(VIM)

## Warning: package 'VIM' was built under R version 4.3.2

## Zorunlu paket yükleniyor: colorspace

## Zorunlu paket yükleniyor: grid

## VIM is ready to use.

## Suggestions and bug-reports can be submitted at:
https://github.com/statistikat/VIM/issues

##
## Attaching package: 'VIM'

## The following object is masked from 'package:datasets':
##
##     sleep

# Column-wise missing value count
colSums(is.na(new_data))

##     time      trt      age     wtkg     hemo     homo    drugs   karnof
##        0        0        0        0        0        0        0        0
##   oprior      z30  preanti     race   gender     str2    strat  symptom
##        0        0        0        0        0        0        0        0
##    treat   offtrt     cd40    cd420     cd80    cd820 infected
##        0        0        0        0        0        0        0

aggr(new_data, numbers=TRUE, prop=FALSE, combined=TRUE)

## Warning in plot.aggr(res, ...): not enough horizontal space to display
## frequencies
```



- Distribution of the infected target :

```
# Load the necessary libraries
library(ggplot2)

# Calculate the proportions of the infected variable
infected_counts <- table(new_data$infected) / nrow(new_data)

# Visualize the distribution of the infected variable with a pie chart
ggplot(as.data.frame(infected_counts), aes(x = "", y = Freq, fill =
factor(Var1))) +
  geom_bar(stat = "identity", width = 1, color = "white") +  # Create the bar
for the pie chart
  coord_polar("y", start = 0) +  # Convert the bar chart into a pie chart
  theme_minimal() +  # Use a minimal theme for the plot
  scale_fill_manual(values = c("#008b8b", "#8b636c"), name = "Infected",
labels = c("0" = "not infected with AIDS", "1" = "infected with AIDS")) +  #
Customize the colors and legend
  labs(title = "Distribution of Infected Variable",  # Set the title
       x = "",  # Remove x-axis label
       y = "") +  # Remove y-axis label
  theme(axis.text.x = element_blank(),  # Remove x-axis text
        axis.ticks = element_blank(),  # Remove axis ticks
        panel.grid = element_blank())  # Remove grid lines
```

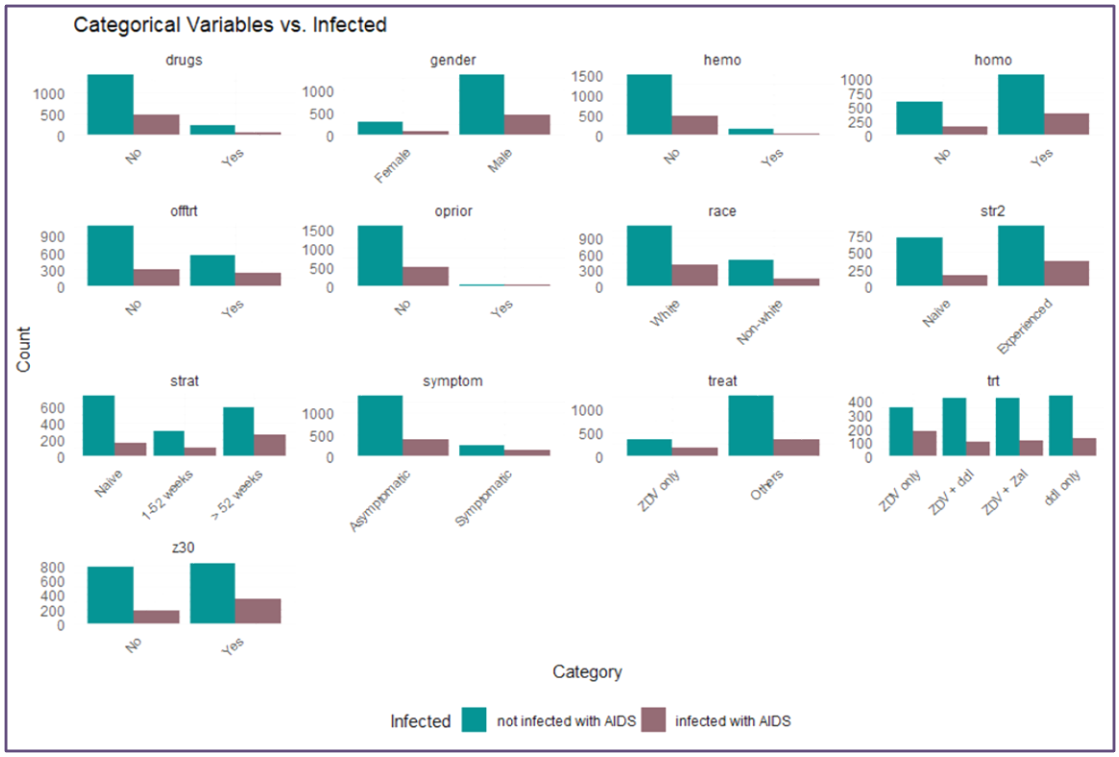


- We can see that mostly patients are not infected with AIDS which is a good thing.
- Distribution of the categorical variable according to the infected variable

```
library(tidyverse)

## Warning: package 'tidyr' was built under R version 4.3.3

## Warning: package 'purrr' was built under R version 4.3.3

## Warning: package 'dplyr' was built under R version 4.3.3

## Warning: package 'stringr' was built under R version 4.3.3

## Warning: package 'lubridate' was built under R version 4.3.3

## ── Attaching core tidyverse packages ──────────────────────── tidyverse
2.0.0 ──
## ✓ dplyr     1.1.4     ✓ stringr   1.5.1
## ✓ forcats   1.0.0     ✓ tibble    3.2.1
## ✓ lubridate 1.9.3     ✓ tidyr     1.3.1
## ✓ purrr     1.0.4
## ── Conflicts ────────────────────────────────────────
tidyverse_conflicts() ──
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

```r
library(dplyr)
library(tidyr)

# Create a new data frame called new_data1
new_data1 <- new_data

# Convert the "infected" variable to a factor (if not already)
new_data1$infected <- factor(new_data1$infected)

# Label the categorical variables
new_data1 <- new_data1 %>%
  mutate(
    trt = factor(trt, levels = c(0, 1, 2, 3), labels = c("ZDV only", "ZDV +
ddI", "ZDV + Zal", "ddI only")),
    hemo = factor(hemo, levels = c(0, 1), labels = c("No", "Yes")),
    homo = factor(homo, levels = c(0, 1), labels = c("No", "Yes")),
    drugs = factor(drugs, levels = c(0, 1), labels = c("No", "Yes")),
    oprior = factor(oprior, levels = c(0, 1), labels = c("No", "Yes")),
    z30 = factor(z30, levels = c(0, 1), labels = c("No", "Yes")),
    race = factor(race, levels = c(0, 1), labels = c("White", "Non-white")),
    gender = factor(gender, levels = c(0, 1), labels = c("Female", "Male")),
    str2 = factor(str2, levels = c(0, 1), labels = c("Naive",
"Experienced")),
    strat = factor(strat, levels = c(1, 2, 3), labels = c("Naive", "1-52
weeks", "> 52 weeks")),
    symptom = factor(symptom, levels = c(0, 1), labels = c("Asymptomatic",
"Symptomatic")),
    treat = factor(treat, levels = c(0, 1), labels = c("ZDV only",
"Others")),
    offtrt = factor(offtrt, levels = c(0, 1), labels = c("No", "Yes"))
  )

# Select the categorical variables
categorical_vars <- new_data1 %>% select(trt, hemo, homo, drugs, oprior, z30,
race, gender, str2, strat, symptom, treat, offtrt)

# Examine the relationship between categorical variables and "infected"
new_data1 %>%
  select(infected, everything()) %>%
  pivot_longer(cols = all_of(names(categorical_vars)), names_to = "variable",
values_to = "value") %>%
  ggplot(aes(x = value, fill = factor(infected))) +
  geom_bar(position = "dodge") +
  facet_wrap(~ variable, scales = "free") +
  scale_fill_manual(values = c("#008b8b", "#8b636c"), name = "Infected",
labels = c("0" = "not infected with AIDS", "1" = "infected with AIDS")) +  #
Customize colors
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 8)) +  # Set
axis text to 45 degrees tilt and size 8pt
```

```
  labs(title = "Categorical Variables vs. Infected", x = "Category", y =
"Count") +
  theme(legend.position = "bottom")  # Move the legend to the bottom
```

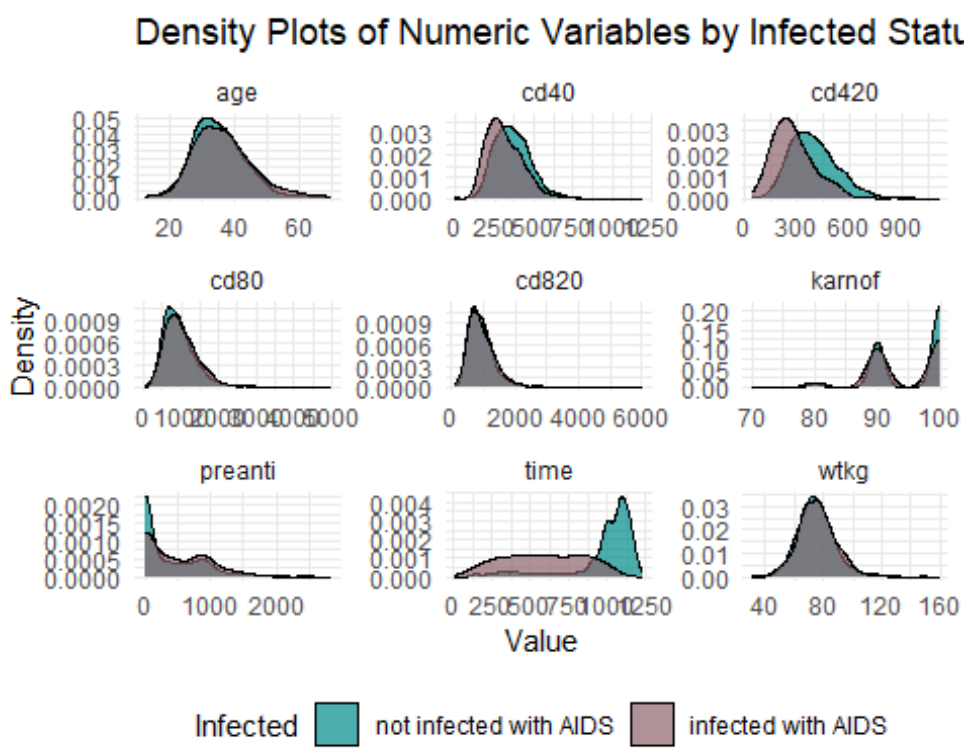


- This chart breaks down categorical variables by infection status (0 for uninfected, 1 for infected). Key observations include a higher count of infected individuals among drug users and those with prior/off treatments. Symptomatic presentation also correlates with a higher infection rate. Gender and certain treatment types show relatively similar counts across infection statuses. While race and strategy duration exhibit differences in infection rates, variables like hemo, homo, and z30 don't show significant associations with infection status. These insights provide valuable cues for understanding factors linked to infection rates and guiding targeted interventions.

- Distribution of numeric variables according to infected

```
# Select numeric variables and include the infected variable
numeric_vars <- new_data %>% select(time, age, wtkg, cd40, cd420, cd80,
cd820, karnof, preanti, infected)

# Analyze relationships between numeric variables and "infected" using
density plots
numeric_vars %>%
  gather(key = "variable", value = "value", -infected) %>%
  ggplot(aes(x = value, fill = factor(infected))) +
  geom_density(alpha = 0.7) +
  facet_wrap(~ variable, scales = "free") +  # Create a facet plot for each
```

```
variable
  scale_fill_manual(values = c("#008b8b", "#8b636c"), name = "Infected",
labels = c("0" = "not infected with AIDS", "1" = "infected with AIDS")) +
  scale_y_continuous(labels = scales::comma) +  # Format y-axis labels as
regular numbers
  theme_minimal() +
  labs(title = "Density Plots of Numeric Variables by Infected Status",
       x = "Value",
       y = "Density",
       fill = "Infected") +
  theme(legend.position = "bottom")  # Move the legend to the bottom
```



- This visualization illustrates density plots of numeric variables categorized by infection status (0 for uninfected, 1 for infected). While age distributions show similarity between the groups, CD marker levels like cd40 and cd420 differ slightly, suggesting lower levels in infected individuals. Karnof levels exhibit a distinct pattern, with infected individuals showing higher values. Variables such as preanti and time display marked differences, especially with "time" showing a bi-modal distribution in infected individuals. Weight distribution, represented by wtkg, also differs, with infected individuals having weight concentrated in a narrower range. Overall, this visualization helps identify factors associated with infection status.

- Observing the correlation relationship.

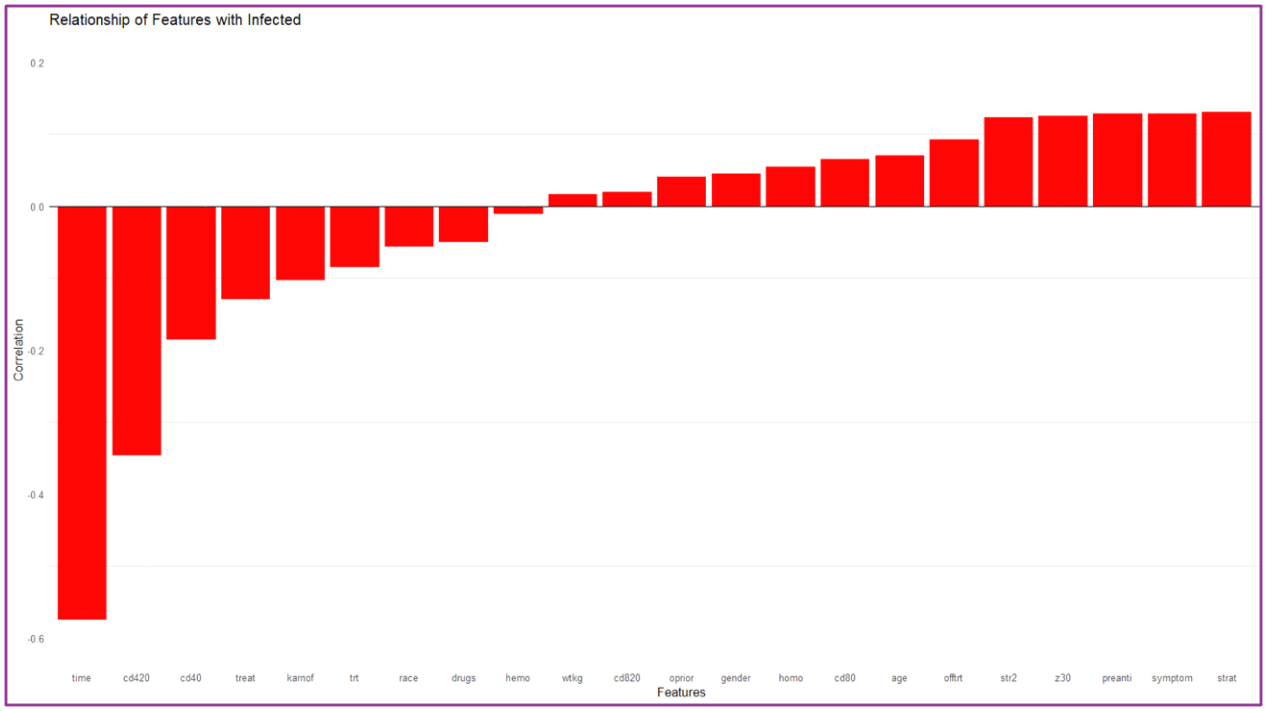
```
# Load required libraries (if not already loaded)
library(tidyverse)
```

```r
# Select only numerical columns
num_cols <- select_if(new_data, is.numeric)

# Calculate correlations
infect_corr <- cor(num_cols)

# Filter for correlations with "infected" column
infect_corr <- infect_corr["infected", ]

# Sort correlations in descending order
infect_corr <- sort(infect_corr, decreasing = TRUE)

# Print the correlations
print(infect_corr)
```

```
##    infected       strat     symptom     preanti         z30        str2
##  1.00000000  0.13144146  0.12923368  0.12845296  0.12548824  0.12339005
##      offtrt         age        cd80        homo      gender      oprior
##  0.09285043  0.07038376  0.06533828  0.05427855  0.04511811  0.04125079
##       cd820        wtkg        hemo       drugs        race         trt
##  0.01949383  0.01620467 -0.01115356 -0.04979400 -0.05597656 -0.08435965
##      karnof       treat        cd40       cd420        time
## -0.10294383 -0.12955957 -0.18564666 -0.34590772 -0.57498872
```

```r
# Set plot aesthetics
ggplot(data.frame(features = names(infect_corr), correlation = infect_corr),
       aes(x = reorder(features, correlation), y = correlation)) +
  # Use geom_bar for bar plot, bars colored in red
  geom_bar(stat = "identity", fill = "red") +
  # Add a horizontal line at y = 0 (solid line)
  geom_hline(yintercept = 0, linetype = "solid") +
  # Rotate x-axis labels
  theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
  # Set labels and title
  labs(x = "Features", y = "Correlation", title = "Relationship of Features
with Infected") +
  # Set minimal theme
  theme_minimal() +
  # Set y-axis limits
  ylim(-0.6, 0.2) +
  # Remove extra grid lines and adjust appearance
  theme(panel.grid.major = element_blank(),
        panel.grid.minor = element_line(colour = "lightgray", linetype =
"solid"))
```

```
## Warning: Removed 1 row containing missing values or values outside the
scale range
## (`geom_bar()`).
```



- This bar chart shows the relationship between various features and the "infected" status, indicating how likely someone is to be infected based on different factors. Longer durations, lower levels of certain CD markers, and the presence of pre-antibodies or symptoms are associated with higher infection rates. Some features, like age and treatment, show weak correlations, while others like karnof and race have moderate correlations. This chart helps identify potential predictors of infection, guiding further analysis and intervention strategies.

MACHINE LEARNING MODELS

- Data split for our data set:

```
library(rsample)

## Warning: package 'rsample' was built under R version 4.3.3

set.seed(25)
data_split <- initial_split(data = new_data, prop = 0.80)
data_train <- data_split |> training()
data_test <- data_split |> testing()
```

- 4 vanilla models were created.

LOGİSTİC MODEL

```
library(caret)

## Warning: package 'caret' was built under R version 4.3.2

## Zorunlu paket yükleniyor: lattice

## Warning: package 'lattice' was built under R version 4.3.3
```

```
##
## Attaching package: 'caret'

## The following object is masked from 'package:purrr':
##
##     lift
```

```r
model_lr <- glm(infected ~. , data = data_train, family = "binomial")
predicted_probs_lr <- predict(model_lr, data_test, type = "response")
predicted_classes_lr <- ifelse(predicted_probs_lr > 0.5, 1, 0)

confusionMatrix(table(ifelse(data_test$infected == "1", "1", "0"),
                      predicted_classes_lr),
                positive = "1")
```

```
## Confusion Matrix and Statistics
##
##    predicted_classes_lr
##       0   1
##   0 302  23
##   1  36  67
##
##                Accuracy : 0.8621
##                  95% CI : (0.8258, 0.8934)
##     No Information Rate : 0.7897
##     P-Value [Acc > NIR] : 7.673e-05
##
##                   Kappa : 0.6058
##
##  Mcnemar's Test P-Value : 0.1182
##
##             Sensitivity : 0.7444
##             Specificity : 0.8935
##          Pos Pred Value : 0.6505
##          Neg Pred Value : 0.9292
##              Prevalence : 0.2103
##          Detection Rate : 0.1565
##    Detection Prevalence : 0.2407
##       Balanced Accuracy : 0.8190
##
##        'Positive' Class : 1
##
```

```r
#train
predicted_probs_lr_t <- predict(model_lr, data_train, type = "response")
predicted_classes_lr_t <- ifelse(predicted_probs_lr_t > 0.5, 1, 0)

confusionMatrix(table(ifelse(data_train$infected == "1", "1", "0"),
                      predicted_classes_lr_t),
                positive = "1")
```

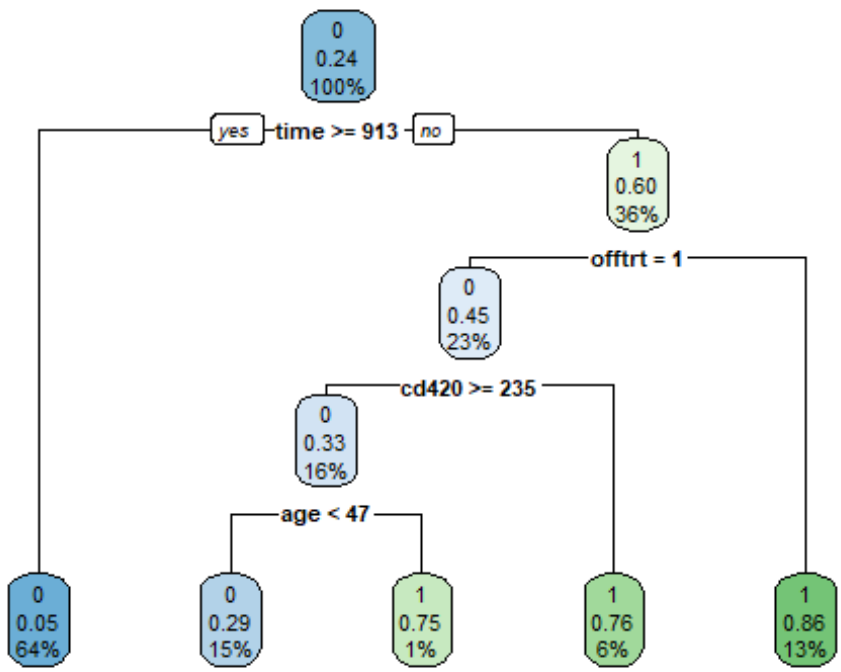
```
## Confusion Matrix and Statistics
##
##    predicted_classes_lr_t
##        0    1
##   0 1226   67
##   1  157  261
##
##                Accuracy : 0.8691
##                  95% CI : (0.8522, 0.8847)
##     No Information Rate : 0.8083
##     P-Value [Acc > NIR] : 1.457e-11
##
##                   Kappa : 0.6176
##
##  Mcnemar's Test P-Value : 2.738e-09
##
##             Sensitivity : 0.7957
##             Specificity : 0.8865
##          Pos Pred Value : 0.6244
##          Neg Pred Value : 0.9482
##              Prevalence : 0.1917
##          Detection Rate : 0.1525
##    Detection Prevalence : 0.2443
##       Balanced Accuracy : 0.8411
##
##        'Positive' Class : 1
##
```

DECİSİON TREE MODEL

```
library(rpart.plot)

## Warning: package 'rpart.plot' was built under R version 4.3.3

## Zorunlu paket yükleniyor: rpart

## Warning: package 'rpart' was built under R version 4.3.3

library(rpart)
data_train$infected <- as.factor(data_train$infected)
data_test$infected <- as.factor(data_test$infected)

model_dt <- rpart(infected ~ .,
                  data = data_train,
                  method = "class")

model_dt

## n= 1711
##
## node), split, n, loss, yval, (yprob)
##       * denotes terminal node
```

```
##
##  1) root 1711 418 0 (0.75569842 0.24430158)
##    2) time>=912.5 1101  51 0 (0.95367847 0.04632153) *
##    3) time< 912.5 610 243 1 (0.39836066 0.60163934)
##      6) offtrt>=0.5 386 175 0 (0.54663212 0.45336788)
##       12) cd420>=235 277  92 0 (0.66787004 0.33212996)
##         24) age< 46.5 253  74 0 (0.70750988 0.29249012) *
##         25) age>=46.5 24   6 1 (0.25000000 0.75000000) *
##       13) cd420< 235 109  26 1 (0.23853211 0.76146789) *
##      7) offtrt< 0.5 224  32 1 (0.14285714 0.85714286) *
```

```
rpart.plot(model_dt)
```




```
# performance metrics of the aids dt
preds_dt <- predict(model_dt, data_test, type = "class")
confusionMatrix(preds_dt, data_test$infected, positive = "1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 308  22
##          1  17  81
##
##                Accuracy : 0.9089
##                  95% CI : (0.8775, 0.9344)
##     No Information Rate : 0.7593
```

```
##     P-Value [Acc > NIR] : 1.171e-15
##
##                   Kappa : 0.7465
##
##  Mcnemar's Test P-Value : 0.5218
##
##             Sensitivity : 0.7864
##             Specificity : 0.9477
##          Pos Pred Value : 0.8265
##          Neg Pred Value : 0.9333
##              Prevalence : 0.2407
##          Detection Rate : 0.1893
##    Detection Prevalence : 0.2290
##       Balanced Accuracy : 0.8671
##
##        'Positive' Class : 1
##
```

```r
#train
preds_dt<- predict(model_dt, data_train, type = "class")
#df_train$Class <- factor(df_train$Class, levels = c("0", "1"))

confusionMatrix(preds_dt,
                data_train$infected,
                positive = "1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1229  125
##          1   64  293
##
##                Accuracy : 0.8895
##                  95% CI : (0.8737, 0.904)
##     No Information Rate : 0.7557
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.6853
##
##  Mcnemar's Test P-Value : 1.275e-05
##
##             Sensitivity : 0.7010
##             Specificity : 0.9505
##          Pos Pred Value : 0.8207
##          Neg Pred Value : 0.9077
##              Prevalence : 0.2443
##          Detection Rate : 0.1712
##    Detection Prevalence : 0.2086
##       Balanced Accuracy : 0.8257
```

```
##
##        'Positive' Class : 1
##
```

BAGGING TREE MODEL

```
set.seed(258)
library(ranger)

## Warning: package 'ranger' was built under R version 4.3.3

model_bt <- ranger(infected~.,
                   data_train,
                   mtry = 22)

preds_bt <- predict(model_bt, data = data_test, type =
"response")$predictions

confusionMatrix(preds_bt,
                data_test$infected,
                positive = "1")

## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 307  24
##          1  18  79
##
##                Accuracy : 0.9019
##                  95% CI : (0.8697, 0.9284)
##     No Information Rate : 0.7593
##     P-Value [Acc > NIR] : 3.288e-14
##
##                   Kappa : 0.7261
##
##  Mcnemar's Test P-Value : 0.4404
##
##             Sensitivity : 0.7670
##             Specificity : 0.9446
##          Pos Pred Value : 0.8144
##          Neg Pred Value : 0.9275
##              Prevalence : 0.2407
##          Detection Rate : 0.1846
##    Detection Prevalence : 0.2266
##       Balanced Accuracy : 0.8558
##
##        'Positive' Class : 1
##

#train
preds_bt <- predict(model_bt, data = data_train, type =
```

```
"response")$predictions

confusionMatrix(preds_bt,
                data_train$infected,
                positive = "1")

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1293    0
##          1    0  418
##
##                Accuracy : 1
##                  95% CI : (0.9978, 1)
##     No Information Rate : 0.7557
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
##             Sensitivity : 1.0000
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 1.0000
##              Prevalence : 0.2443
##          Detection Rate : 0.2443
##    Detection Prevalence : 0.2443
##       Balanced Accuracy : 1.0000
##
##        'Positive' Class : 1
##
```

RANDOM FOREST MODEL

```
set.seed(258)
model_rf <- ranger(infected ~.,
                   data = data_train)

preds_rf <- predict(model_rf, data_test, type ="response")$predictions
confusionMatrix(preds_rf,
                data_test$infected,
                positive = "1")

## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 307  22
##          1  18  81
```

```
##
##                Accuracy : 0.9065
##                  95% CI : (0.8749, 0.9324)
##     No Information Rate : 0.7593
##     P-Value [Acc > NIR] : 3.657e-15
##
##                   Kappa : 0.7408
##
##  Mcnemar's Test P-Value : 0.6353
##
##             Sensitivity : 0.7864
##             Specificity : 0.9446
##          Pos Pred Value : 0.8182
##          Neg Pred Value : 0.9331
##              Prevalence : 0.2407
##          Detection Rate : 0.1893
##    Detection Prevalence : 0.2313
##       Balanced Accuracy : 0.8655
##
##        'Positive' Class : 1
##
```

```r
#train
preds_rf <- predict(model_rf, data_train, type ="response")$predictions
confusionMatrix(preds_rf,
                data_train$infected,
                positive = "1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1293    0
##          1    0  418
##
##                Accuracy : 1
##                  95% CI : (0.9978, 1)
##     No Information Rate : 0.7557
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
##             Sensitivity : 1.0000
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 1.0000
##              Prevalence : 0.2443
##          Detection Rate : 0.2443
```

```
##    Detection Prevalence : 0.2443
##       Balanced Accuracy : 1.0000
##
##        'Positive' Class : 1
##
```

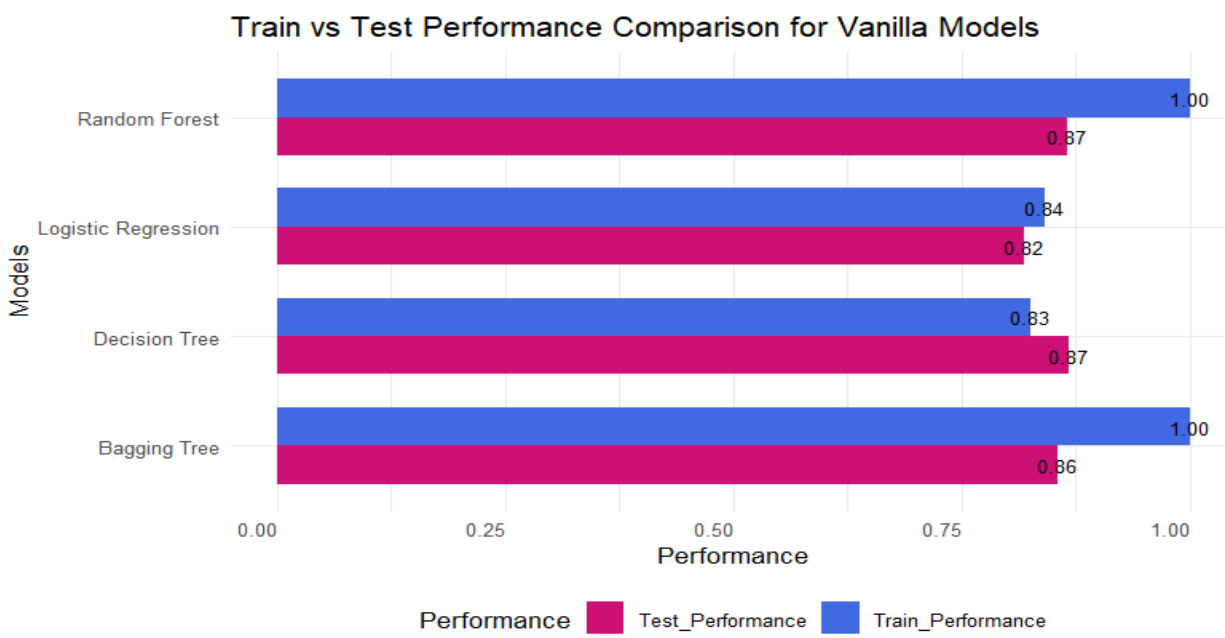
CONCLUSION:

Based on the comparison of four machine learning models (Logistic Regression, Decision Tree, Bagging Tree, and Random Forest) on the test dataset, the following observations can be made: - Logistic Regression: The model achieved an accuracy of approximately 86.21% on the test data, with a sensitivity of 74.44% and a specificity of 89.35%. It correctly classified 67 out of 91 infected cases but had a relatively lower positive predictive value of 65.05%. - Decision Tree: The decision tree model performed slightly better than logistic regression, with an accuracy of about 90.89% on the test data. It demonstrated a sensitivity of 78.64% and a specificity of 94.77%. The positive predictive value was 82.65%, indicating a better performance in predicting positive cases. - Bagging Tree: The bagging tree model showed similar performance to the decision tree, with an accuracy of approximately 90.19% on the test data. It exhibited a sensitivity of 76.70% and a specificity of 94.46%. The positive predictive value was 81.44%, indicating robustness in predicting positive cases. - Random Forest: Similar to the bagging tree model, random forest achieved an accuracy of around 90.65% on the test data. It showed a sensitivity of 78.64% and a specificity of 94.46%. The positive predictive value was 81.82%. Overall, all models demonstrated strong performance in predicting infection status, with decision tree-based models (Decision Tree, Bagging Tree, and Random Forest) outperforming logistic regression in terms of accuracy and positive predictive value. However, the decision tree-based models did not exhibit significant differences in performance among themselves.

- Visualisation of train and test performance of models:

```r
library(dplyr)

# Calculate model performance metrics

ModelPerformance <- data.frame(
  Model = c("Logistic Regression", "Decision Tree", "Bagging Tree", "Random
Forest"),
  Train_Performance = c(0.8411, 0.8257, 1, 1),  # Example values, should be
replaced with actual data
  Test_Performance = c(0.8190, 0.8671, 0.8558, 0.8655)    # Example values,
should be replaced with actual data
)
# Sort values from largest to smallest

ModelPerformance_long <- pivot_longer(ModelPerformance, cols =
c(Train_Performance, Test_Performance),
                                      names_to = "Performance", values_to =
"Value")
```

```r
# Draw a bar plot (all bars will be in pink color and there will be no
legend)
# Draw a bar plot
library(ggplot2)  # Draw the graph
ggplot(ModelPerformance_long, aes(x = Model, y = Value, fill = Performance))
+
  geom_bar(stat = "identity", position = "dodge", width = 0.7) +
  geom_text(aes(label = sprintf("%.2f", Value)),
            position = position_dodge(width = 0.7),
            vjust = 0.5, size = 3, angle = 0, color = "black") +  # Show the
value inside each bar
  labs(title = "Train vs Test Performance Comparison for Vanilla Models",
       x = "Models",
       y = "Performance",
       fill = "Performance") +
  scale_fill_manual(values = c("Train_Performance" = "#4169e1",
"Test_Performance" = "deeppink3")) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 0, hjust = 1)) +
  theme(legend.position = "bottom") +  # Place the legend at the bottom
  coord_flip()
```

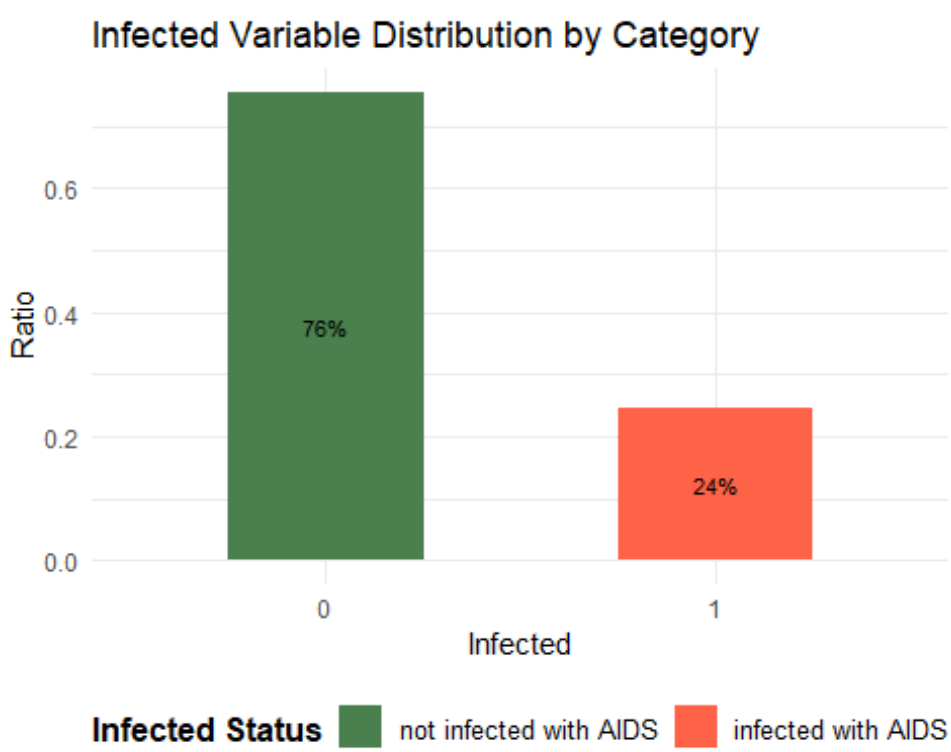


- The bar chart compares the training and testing accuracy of four machine learning models: Random Forest, Logistic Regression, Decision Tree, and Bagging Tree. Random Forest and Bagging Tree exhibit signs of overfitting, with perfect training accuracy but reduced testing accuracy, while Logistic Regression and Decision Tree models maintain consistent performance between training and testing datasets. This suggests that simpler models like Logistic Regression and Decision Tree may be more reliable for general predictions on this dataset, whereas ensemble models like Random Forest and Bagging Tree might require adjustments to prevent overfitting.

Overall, the visualization provides insights into the performance and potential adjustments needed for each model to ensure better generalization.

- Detection of imbalance in the data set

```
# Load necessary libraries
library(ggplot2)

# Calculate proportions of the "infected" variable in the dataset
infected_counts <- table(data_train$infected) / dim(data_train)[1]

# Create a bar plot
ggplot(data = as.data.frame(infected_counts), aes(x = factor(Var1), y = Freq,
fill = factor(Var1))) +
  geom_bar(stat = "identity", width = 0.5) +
  geom_text(aes(label = scales::percent(Freq)), position =
position_stack(vjust = 0.5), size = 3) +  # Display values on top of bars
  labs(title = "Infected Variable Distribution by Category",
       x = "Infected",
       y = "Ratio") +
  scale_fill_manual(name = "Infected Status", labels = c("0" = "not infected
with AIDS", "1" = "infected with AIDS"), values = c("0" = "#4a804d", "1" =
"#ff6347")) +  # Legend labels and colors
  theme_minimal() +
  theme(legend.position = "bottom", legend.title = element_text(size = 12,
face = "bold"), legend.text = element_text(size = 10))  # Legend settings
```



- The graph shows that the majority of people (76%) are not infected with AIDS. However, a significant minority (24%) are infected with the disease. This highlights the importance of HIV/AIDS prevention and testing. But we can clearly see that there is a big imbalanced problem here. So we have to work on this problem.

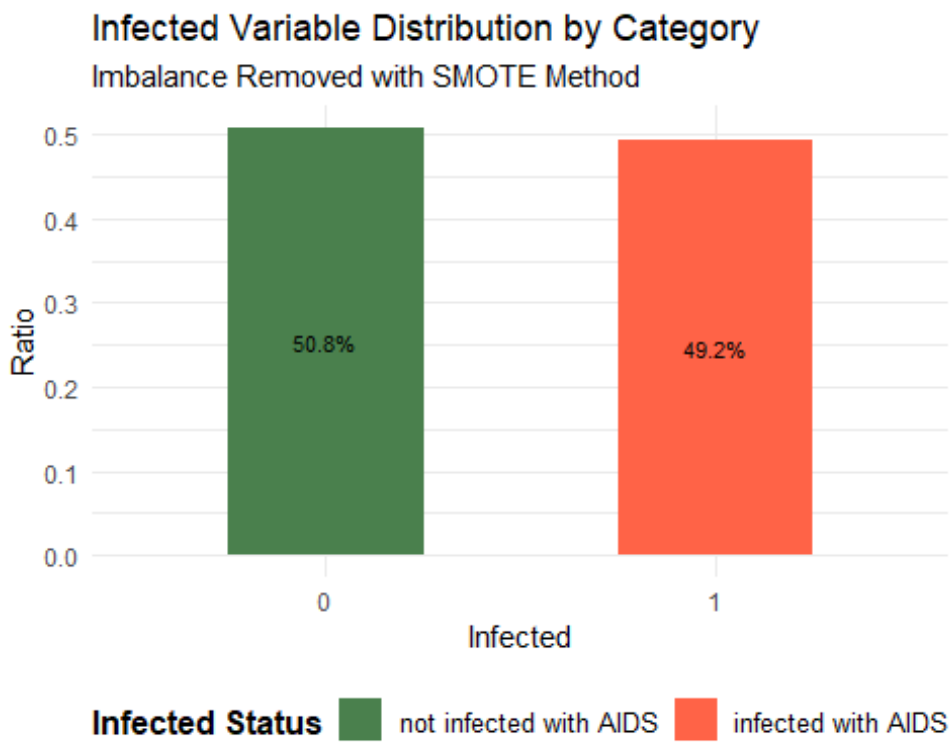
- Let's solve the imbalance problem with SMOTE

```
# Load necessary libraries
library(caret)
library(randomForest)

## Warning: package 'randomForest' was built under R version 4.3.2

## randomForest 4.7-1.1

## Type rfNews() to see new features/changes/bug fixes.

##
## Attaching package: 'randomForest'

## The following object is masked from 'package:ranger':
##
##     importance

## The following object is masked from 'package:dplyr':
##
##     combine

## The following object is masked from 'package:ggplot2':
##
##     margin

library(smotefamily)

## Warning: package 'smotefamily' was built under R version 4.3.3

# Balance with SMOTE
set.seed(42)
data_train_smote <- SMOTE(data_train[,-ncol(data_train)],
data_train$infected)$data
colnames(data_train_smote)[ncol(data_train_smote)] <- "infected"

data_train_smote

## # A tibble: 2,547 × 23
##     time   trt   age  wtkg  hemo  homo drugs karnof oprior   z30 preanti
race
##    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>  <dbl> <dbl>   <dbl>
<dbl>
##  1   322     1    47  94       0     1     0    100      0     1    1421
0
##  2   470     1    27  79.4     0     1     0     90      0     1      55
```

```
0
##  3   628     1    32  90.5     0     0     0     90      0     0       0
0
##  4   581     1    30  67.4     0     1     0    100      0     0       0
0
##  5   190     1    51 119.      0     1     0    100      0     0       0
0
##  6   318     1    34  68.3     0     1     0     90      0     1     980
0
##  7   281     1    34  73.8     0     1     0    100      0     0       0
0
##  8   627     0    62  76.4     0     1     0     90      0     0       0
0
##  9   896     0    30  81.6     0     0     0    100      0     0     389
0
## 10   749     2    29  66.7     0     0     0    100      0     0       0
1
## # ℹ 2,537 more rows
## # ℹ 11 more variables: gender <dbl>, str2 <dbl>, strat <dbl>, symptom
<dbl>,
## #   treat <dbl>, offtrt <dbl>, cd40 <dbl>, cd420 <dbl>, cd80 <dbl>,
## #   cd820 <dbl>, infected <chr>
```

- Let's see the distribution after the imbalance problem is fixed:

```
# Calculate proportions of the "infected" variable in the balanced dataset
proportions <- table(data_train_smote$infected) / dim(data_train_smote)[1]

# Create a bar plot
ggplot(data = as.data.frame(proportions), aes(x = factor(Var1), y = Freq,
fill = factor(Var1))) +
  geom_bar(stat = "identity", width = 0.5) +
  geom_text(aes(label = scales::percent(Freq)), position =
position_stack(vjust = 0.5), size = 3) +  # Display values on top of bars
  labs(title = "Infected Variable Distribution by Category",
       subtitle = "Imbalance Removed with SMOTE Method",
       x = "Infected",
       y = "Ratio") +
  scale_fill_manual(name = "Infected Status", labels = c("0" = "not infected
with AIDS", "1" = "infected with AIDS"), values = c("0" = "#4a804d", "1" =
"#ff6347")) +  # Legend labels and colors
  theme_minimal() +
  theme(legend.position = "bottom", legend.title = element_text(size = 12,
face = "bold"), legend.text = element_text(size = 10))  # Legend settings
```



- Here, the performance of vanilla models (Logistic Regression, Decision Tree, Bagging Tree, Random Forest) is evaluated after the imbalance is removed using SMOTE:

```
# Convert 'infected' variable to factor in the balanced dataset
data_train_smote$infected <- as.factor(data_train_smote$infected)

### Logistic Regression
# Train logistic regression model
model_lr <- glm(infected ~ . , data = data_train_smote, family = "binomial")

# Predict on test set
predicted_probs_lr <- predict(model_lr, data_test, type = "response")
predicted_classes_lr <- ifelse(predicted_probs_lr > 0.5, 1, 0)

# Confusion matrix and performance metrics on test set
confusionMatrix(table(ifelse(data_test$infected == "1", "1", "0"),
                      predicted_classes_lr),
                positive = "1")

## Confusion Matrix and Statistics
##
##    predicted_classes_lr
##       0   1
##   0 281  44
##   1  11  92
##
```

```
##                Accuracy : 0.8715
##                  95% CI : (0.836, 0.9017)
##     No Information Rate : 0.6822
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.6831
##
##  Mcnemar's Test P-Value : 1.597e-05
##
##             Sensitivity : 0.6765
##             Specificity : 0.9623
##          Pos Pred Value : 0.8932
##          Neg Pred Value : 0.8646
##              Prevalence : 0.3178
##          Detection Rate : 0.2150
##    Detection Prevalence : 0.2407
##       Balanced Accuracy : 0.8194
##
##        'Positive' Class : 1
##
```

```r
# Predict on train set for comparison (detect overfitting)
predicted_probs_lr_train <- predict(model_lr, data_train_smote, type =
"response")
predicted_classes_lr_train <- ifelse(predicted_probs_lr_train > 0.5, 1, 0)

# Confusion matrix and performance metrics on train set
confusionMatrix(table(ifelse(data_train_smote$infected == "1", "1", "0"),
                      predicted_classes_lr_train),
                positive = "1")
```

```
## Confusion Matrix and Statistics
##
##    predicted_classes_lr_train
##        0    1
##   0 1140  153
##   1  259  995
##
##                Accuracy : 0.8382
##                  95% CI : (0.8234, 0.8523)
##     No Information Rate : 0.5493
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.676
##
##  Mcnemar's Test P-Value : 2.304e-07
##
##             Sensitivity : 0.8667
##             Specificity : 0.8149
##          Pos Pred Value : 0.7935
```

```
##          Neg Pred Value : 0.8817
##              Prevalence : 0.4507
##          Detection Rate : 0.3907
##    Detection Prevalence : 0.4923
##       Balanced Accuracy : 0.8408
##
##        'Positive' Class : 1
##
```

```r
### Decision Tree
# Train decision tree model
model_dt <- rpart(infected ~ .,
                  data = data_train_smote,
                  method = "class")

# Plot decision tree (optional)
# rpart.plot(model_dt)

# Predict on test set
preds_dt <- predict(model_dt, data_test, type = "class")

# Confusion matrix and performance metrics on test set
confusionMatrix(preds_dt, data_test$infected, positive = "1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 279  15
##          1  46  88
##
##                Accuracy : 0.8575
##                  95% CI : (0.8207, 0.8892)
##     No Information Rate : 0.7593
##     P-Value [Acc > NIR] : 3.632e-07
##
##                   Kappa : 0.6464
##
##  Mcnemar's Test P-Value : 0.0001225
##
##             Sensitivity : 0.8544
##             Specificity : 0.8585
##          Pos Pred Value : 0.6567
##          Neg Pred Value : 0.9490
##              Prevalence : 0.2407
##          Detection Rate : 0.2056
##    Detection Prevalence : 0.3131
##       Balanced Accuracy : 0.8564
##
```

```
##        'Positive' Class : 1
##
```

```r
# Predict on train set for comparison
preds_dt_train <- predict(model_dt, data_train_smote, type = "class")

# Confusion matrix and performance metrics on train set
confusionMatrix(preds_dt_train, data_train_smote$infected, positive = "1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1160  149
##          1  133 1105
##
##                Accuracy : 0.8893
##                  95% CI : (0.8764, 0.9012)
##     No Information Rate : 0.5077
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.7785
##
##  Mcnemar's Test P-Value : 0.3717
##
##             Sensitivity : 0.8812
##             Specificity : 0.8971
##          Pos Pred Value : 0.8926
##          Neg Pred Value : 0.8862
##              Prevalence : 0.4923
##          Detection Rate : 0.4338
##    Detection Prevalence : 0.4861
##       Balanced Accuracy : 0.8892
##
##        'Positive' Class : 1
##
```

```r
### Bagging Tree
# Train bagging tree model
model_bt <- ranger(infected ~ .,
                   data_train_smote,
                   mtry = 22)

# Predict on test set
preds_bt <- predict(model_bt, data = data_test, type =
"response")$predictions

# Confusion matrix and performance metrics on test set
confusionMatrix(preds_bt,
```

```
                data_test$infected,
                positive = "1")

## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 294  21
##          1  31  82
##
##                Accuracy : 0.8785
##                  95% CI : (0.8437, 0.9079)
##     No Information Rate : 0.7593
##     P-Value [Acc > NIR] : 4.426e-10
##
##                   Kappa : 0.6782
##
##  Mcnemar's Test P-Value : 0.212
##
##             Sensitivity : 0.7961
##             Specificity : 0.9046
##          Pos Pred Value : 0.7257
##          Neg Pred Value : 0.9333
##              Prevalence : 0.2407
##          Detection Rate : 0.1916
##    Detection Prevalence : 0.2640
##       Balanced Accuracy : 0.8504
##
##        'Positive' Class : 1
##

# Predict on train set for comparison
preds_bt_train <- predict(model_bt, data = data_train_smote, type =
"response")$predictions

# Confusion matrix and performance metrics on train set
confusionMatrix(preds_bt_train,
                data_train_smote$infected,
                positive = "1")

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1293    0
##          1    0 1254
##
##                Accuracy : 1
##                  95% CI : (0.9986, 1)
##     No Information Rate : 0.5077
##     P-Value [Acc > NIR] : < 2.2e-16
```

```
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
##             Sensitivity : 1.0000
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 1.0000
##              Prevalence : 0.4923
##          Detection Rate : 0.4923
##    Detection Prevalence : 0.4923
##       Balanced Accuracy : 1.0000
##
##        'Positive' Class : 1
##
```

```r
### Random Forest
# Train random forest model
set.seed(258)
model_rf <- ranger(infected ~ .,
                   data = data_train_smote)

# Predict on test set
preds_rf <- predict(model_rf, data_test, type = "response")$predictions

# Confusion matrix and performance metrics on test set
confusionMatrix(preds_rf,
                data_test$infected,
                positive = "1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 300  16
##          1  25  87
##
##                Accuracy : 0.9042
##                  95% CI : (0.8723, 0.9304)
##     No Information Rate : 0.7593
##     P-Value [Acc > NIR] : 1.111e-14
##
##                   Kappa : 0.7455
##
##  Mcnemar's Test P-Value : 0.2115
##
##             Sensitivity : 0.8447
##             Specificity : 0.9231
##          Pos Pred Value : 0.7768
```

```
##          Neg Pred Value : 0.9494
##              Prevalence : 0.2407
##          Detection Rate : 0.2033
##    Detection Prevalence : 0.2617
##       Balanced Accuracy : 0.8839
##
##        'Positive' Class : 1
##
```

```r
# Predict on train set for comparison
preds_rf_train <- predict(model_rf, data_train_smote, type =
"response")$predictions

# Confusion matrix and performance metrics on train set
confusionMatrix(preds_rf_train,
                data_train_smote$infected,
                positive = "1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1293    0
##          1    0 1254
##
##                Accuracy : 1
##                  95% CI : (0.9986, 1)
##     No Information Rate : 0.5077
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
##             Sensitivity : 1.0000
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 1.0000
##              Prevalence : 0.4923
##          Detection Rate : 0.4923
##    Detection Prevalence : 0.4923
##       Balanced Accuracy : 1.0000
##
##        'Positive' Class : 1
##
```

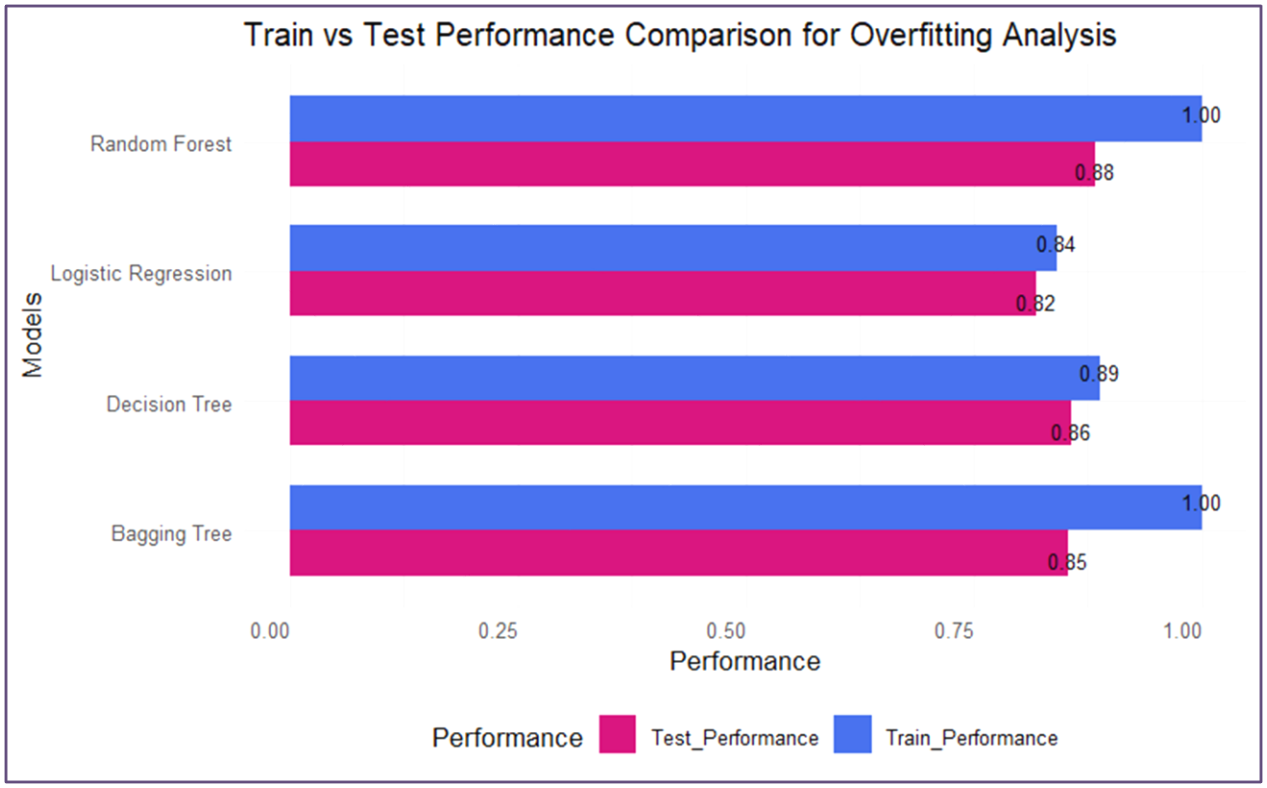
CONCLUSION:

- Logistic Regression and Decision Tree models perform consistently on both training and testing datasets, suggesting robustness without significant overfitting. Although Bagging Tree and Random Forest models show slight signs of overfitting, they still

offer strong predictive capabilities. With further optimization, these models could enhance generalization.

- After addressing imbalance using SMOTE, the performance of various models (Logistic Regression, Decision Tree, Bagging Tree, Random Forest) can be visualized and assessed to detect overfitting.

```
# Convert data to a dataframe
ModelPerformanceK <- data.frame(
  Model = c("Logistic Regression", "Decision Tree", "Bagging Tree", "Random
Forest"),
  Train_Performance = c(0.8408, 0.8892, 1, 1),  # Example values, replace
with actual values
  Test_Performance = c(0.8194, 0.8564, 0.8534, 0.8839)    # Example values,
replace with actual values
)

# Load ggplot2 library
library(ggplot2)

# Reshape data for plotting
ModelPerformanceK_long <- pivot_longer(ModelPerformanceK, cols =
c(Train_Performance, Test_Performance),
                                       names_to = "Performance", values_to =
"Value")

# Create a bar plot
ggplot(ModelPerformanceK_long, aes(x = Model, y = Value, fill = Performance))
+
  geom_bar(stat = "identity", position = "dodge", width = 0.7) +
  geom_text(aes(label = sprintf("%.2f", Value)),
            position = position_dodge(width = 0.9),
            vjust = 0.5, size = 3, angle = 0, color = "black") +
  labs(title = "Train vs Test Performance Comparison for Overfitting
Analysis",
       x = "Models",
       y = "Performance",
       fill = "Performance") +
  scale_fill_manual(values = c("Train_Performance" = "#4169e1",
"Test_Performance" = "deeppink3")) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 0, hjust = 1)) +
  theme(legend.position = "bottom") +  # Place legend at the bottom
  coord_flip()
```



CONCLUSION:

- The bar chart compares training and testing performance of machine learning models, focusing on identifying overfitting. Random Forest's high training accuracy (100%) drops slightly in testing (88%), suggesting potential overfitting. Logistic Regression shows balanced performance between training (84%) and testing (82%) accuracy, indicating a good fit without significant overfitting. The Decision Tree model performs well with minor disparities between training (89%) and testing (86%) accuracy, suggesting moderate overfitting. Bagging Tree, like Random Forest, shows signs of overfitting with perfect training accuracy and reduced testing performance (85%). Ensemble models tend to suffer from overfitting, while Logistic Regression offers a more reliable option with consistent accuracy between training and testing. This comparison emphasizes the importance of balancing accuracy and generalization in model selection for real-world applications.

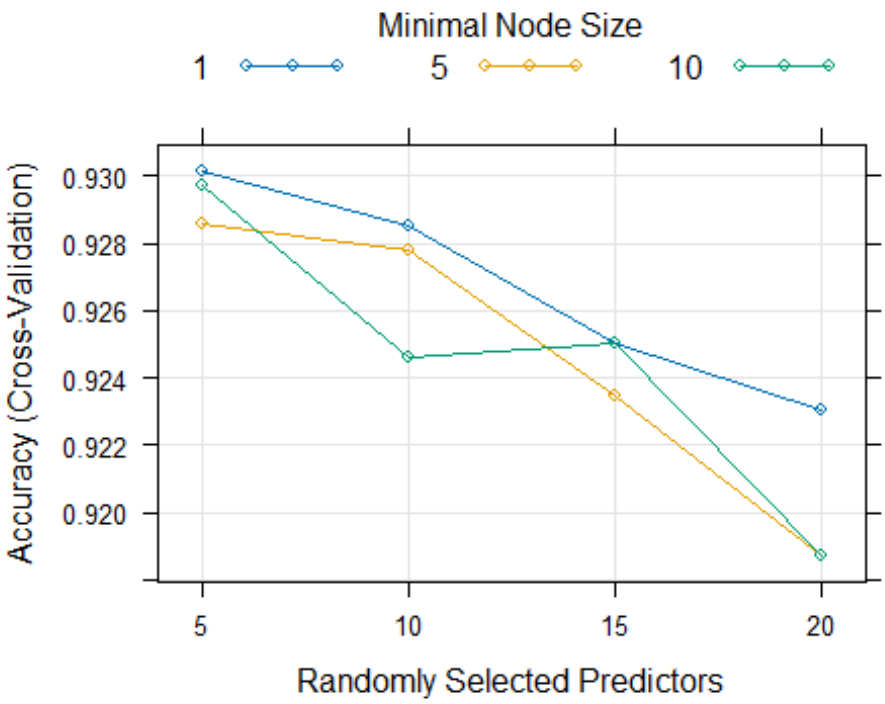
- After resolving the imbalance issue, models were rebuilt. While underfitting was addressed, overfitting still persisted. To mitigate this, cross-validation and hyperparameter tuning were performed, resulting in improved model performance.

```
set.seed(754)
# Create grid for hyperparameter settings
tuneGrid <- expand.grid(.mtry = c(5, 10, 15, 20),
                        .splitrule = c("gini"),
                        .min.node.size = c(1, 5, 10))
```

```r
# Model training with cross-validation
control <- trainControl(method = "cv", number = 10)
model <- train(infected ~ ., data = data_train_smote, method = "ranger",
               trControl = control, tuneGrid = tuneGrid)

# Choosing the best model
best_model <- model$bestTune

#  Printing the best parameters
print(best_model)

##   mtry splitrule min.node.size
## 1    5      gini             1
```

- Visualisation of adjustments made:

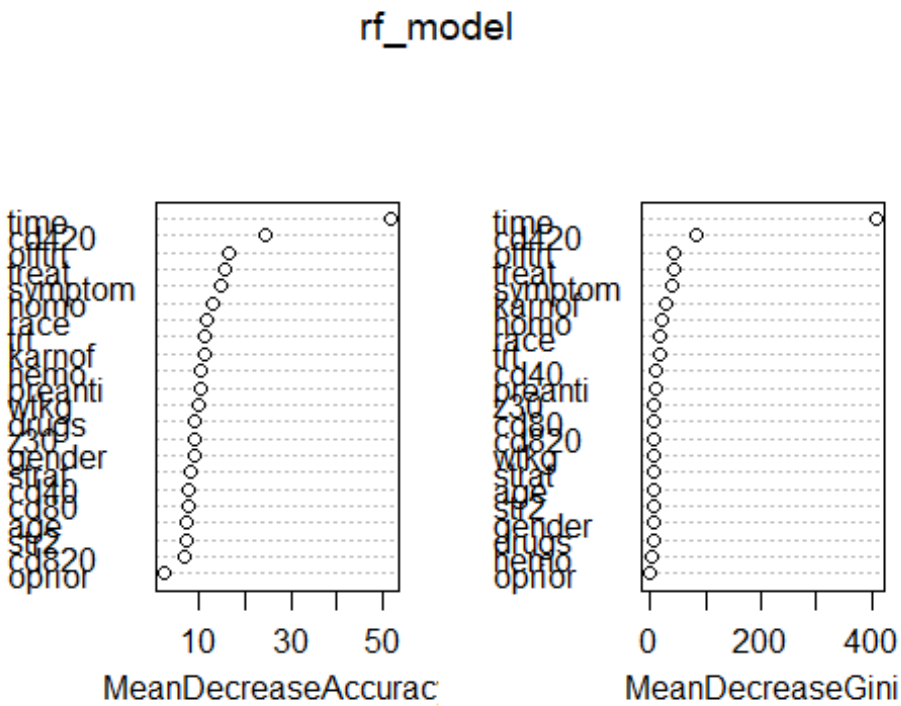
```r
plot(model, xlab = "Randomly Selected Predictors")
```



- The graph shows how the model performs with different settings. Smaller settings are better at first but get worse as things get more complex. Larger settings, while they can handle more stuff, can't tell things apart as well, so they end up less accurate. Adding more predictors at first makes things better, but then it just adds noise. So, to make the model work its best, you need to find the right balance between keeping it simple and letting it be complex.

- The Random Forest model has been retrained.

```r
# Load necessary libraries
library(randomForest)
library(caret)

# Prepare the data
set.seed(24)
data_train$infected <- as.factor(data_train$infected)
data_test$infected <- as.factor(data_test$infected)

# Build the Random Forest model and tune hyperparameters
rf_model <- randomForest(infected ~ .,
                         data = data_train_smote,
                         ntree = 500,       # Number of trees
                         maxnodes = 30,     # Maximum number of nodes
                         mtry = 5,          # Number of variables randomly
sampled at each split
                         splitrule = "gini",# Splitting rule (Gini impurity)
                         min.node.size = 1, # Minimum size of terminal nodes
                         importance = TRUE) # Calculate variable importance

# Print model details
print(rf_model)

##
## Call:
##  randomForest(formula = infected ~ ., data = data_train_smote,      ntree
= 500, maxnodes = 30, mtry = 5, splitrule = "gini",      min.node.size = 1,
importance = TRUE)
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 5
##
##         OOB estimate of  error rate: 9.31%
## Confusion matrix:
##      0    1 class.error
## 0 1171  122  0.09435422
## 1  115 1139  0.09170654

# Plot variable importance
varImpPlot(rf_model)
```




```r
# Make predictions on the test data
preds_rf <- predict(rf_model, data_test)

# Calculate confusion matrix on test data
conf_matrix_rf <- confusionMatrix(preds_rf, data_test$infected, positive =
"1")
print(conf_matrix_rf)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 291  12
##          1  34  91
##
##                Accuracy : 0.8925
##                  95% CI : (0.8593, 0.9202)
##     No Information Rate : 0.7593
##     P-Value [Acc > NIR] : 1.956e-12
##
##                   Kappa : 0.7259
##
##  Mcnemar's Test P-Value : 0.00196
##
##             Sensitivity : 0.8835
##             Specificity : 0.8954
##          Pos Pred Value : 0.7280
```

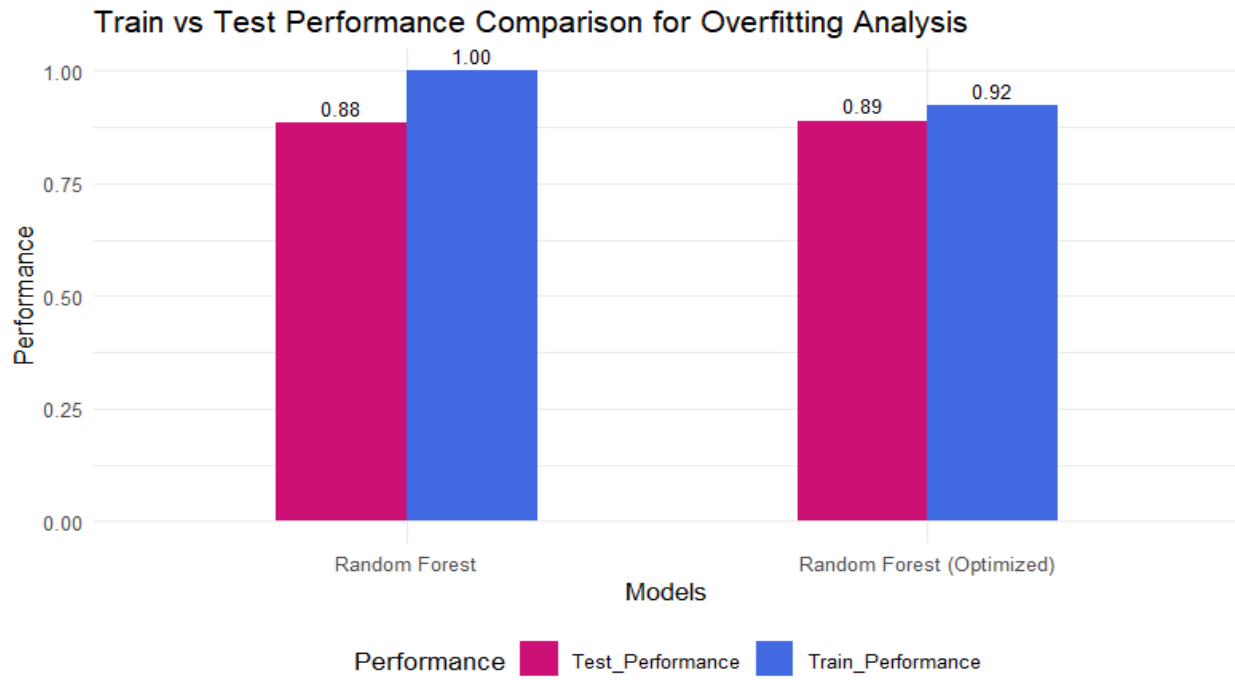
```
##          Neg Pred Value : 0.9604
##              Prevalence : 0.2407
##          Detection Rate : 0.2126
##    Detection Prevalence : 0.2921
##       Balanced Accuracy : 0.8894
##
##        'Positive' Class : 1
##
```

```r
# Make predictions on the training data (for assessing overfitting)
preds_rf_train <- predict(rf_model, data_train_smote)

# Calculate confusion matrix on training data
conf_matrix_rf_train <- confusionMatrix(preds_rf_train,
data_train_smote$infected, positive = "1")
print(conf_matrix_rf_train)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1189   91
##          1  104 1163
##
##                Accuracy : 0.9234
##                  95% CI : (0.9124, 0.9335)
##     No Information Rate : 0.5077
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.8469
##
##  Mcnemar's Test P-Value : 0.3902
##
##             Sensitivity : 0.9274
##             Specificity : 0.9196
##          Pos Pred Value : 0.9179
##          Neg Pred Value : 0.9289
##              Prevalence : 0.4923
##          Detection Rate : 0.4566
##    Detection Prevalence : 0.4974
##       Balanced Accuracy : 0.9235
##
##        'Positive' Class : 1
##
```

- Before optimization, the Random Forest model achieved 100% training and 88.39% test accuracy. After optimization, these figures changed to 92.35% for training and 88.94% for test accuracy. These improvements indicate enhanced generalization capability of the model.

```r
# Convert data to a dataframe
ModelPerformance <- data.frame(
  Model = c("Random Forest", "Random Forest (Optimized)"),
  Train_Performance = c(1, 0.9235),  # Replace with actual values
  Test_Performance = c(0.8839, 0.8894)  # Replace with actual values
)

# Load necessary libraries
library(ggplot2)
library(tidyr)

# Reshape data for plotting
ModelPerformance_long <- pivot_longer(ModelPerformance, cols =
c(Train_Performance, Test_Performance),
                                      names_to = "Performance", values_to =
"Value")

# Create bar plot
ggplot(ModelPerformance_long, aes(x = Model, y = Value, fill = Performance))
+
  geom_bar(stat = "identity", position = "dodge", width = 0.5) +
  geom_text(aes(label = sprintf("%.2f", Value)),
            position = position_dodge(width = 0.5),
            vjust = -0.5, size = 3, angle = 0, color = "black") +  # Display
values inside bars
  labs(title = "Train vs Test Performance Comparison for Overfitting
Analysis",
       x = "Models",
       y = "Performance",
       fill = "Performance") +
  scale_fill_manual(values = c("Train_Performance" = "#4169e1",
"Test_Performance" = "deeppink3")) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 0, hjust = 0.5)) +
  theme(legend.position = "bottom")  # Place legend at the bottom
```

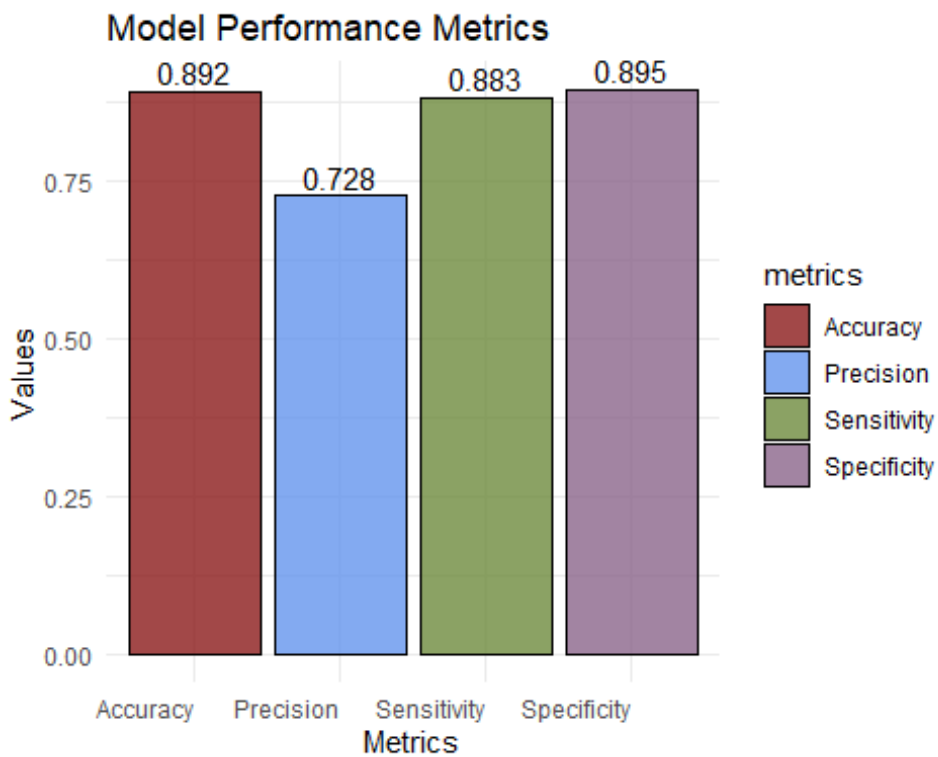


- The bar chart compares the training and testing performances of two versions of the Random Forest model: the standard version and an optimized one. In the standard Random Forest, the training accuracy is perfect at 100%, but the testing accuracy drops to 88%, indicating potential overfitting. However, with the optimized version, although the training accuracy slightly decreases to 92%, the testing accuracy improves to 89%. This suggests that the optimization techniques employed have effectively mitigated overfitting, resulting in a better balance between training and testing performances. Overall, these findings underscore the importance of model optimization in improving generalization, particularly in complex models like Random Forests.

- Other performance metrics of our Random Forest model are analysed and their values are shown:

```
library(ggplot2)

# Performance metrics and values
metrics <- c("Accuracy", "Sensitivity", "Specificity", "Precision")
values <- c(0.8925, 0.8835, 0.8954, 0.7280)

# Convert data to a dataframe
performance_df <- data.frame(metrics, values)

# Create a bar plot
ggplot(performance_df, aes(x = metrics, y = values, fill = metrics)) +
  geom_bar(stat = "identity", color = "black", alpha = 0.8) +
  labs(title = "Model Performance Metrics",
       x = "Metrics",
```

```
      y = "Values") +
  scale_fill_manual(values = c("Accuracy" = "firebrick4",
                               "Sensitivity" = "darkolivegreen4",
                               "Specificity" = "#8b658b",
                               "Precision" = "#6495ed")) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 0, hjust = 1)) +
  geom_text(aes(label = sprintf("%.3f", values)), vjust = -0.3, size = 4)
```

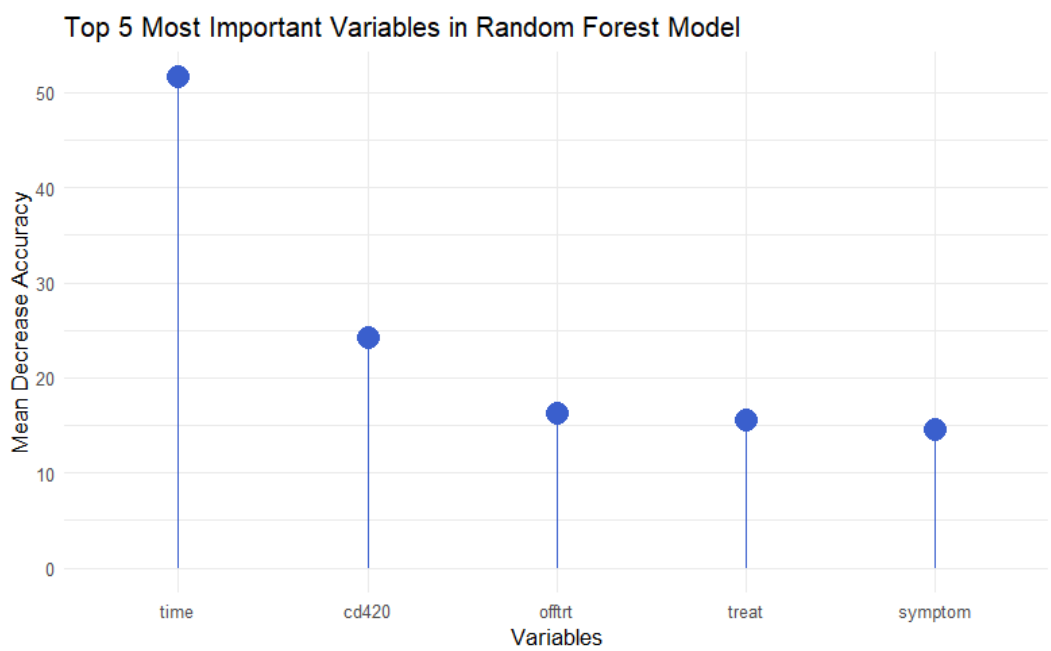


- The bar chart shows the performance of a predictive model, measuring Accuracy, Precision, Sensitivity, and Specificity. Accuracy, at 89.2%, indicates overall correctness, while Precision, at 72.8%, reveals the accuracy of positive predictions. Sensitivity, or recall, at 88.3%, signifies the model's ability to identify true positives, crucial in medical diagnoses. Specificity, at 89.5%, reflects the model's capacity to identify true negatives, vital in avoiding false alarms. Though the model performs well overall, precision could be improved to reduce false positives. Enhancing precision would refine the model's effectiveness, particularly in applications where precise identification is crucial. These metrics offer valuable insights for optimizing the model's performance, ensuring better decision-making in practical settings.

- Lastly, we created a lollipop chart for the top 5 most important variables for our model.

```
# Required libraries
library(randomForest)
library(ggplot2)
```

```r
# Get variable importance scores
importance_values <- importance(rf_model)
importance_df <- as.data.frame(importance_values)
importance_df$Variable <- rownames(importance_df)

# Order by MeanDecreaseAccuracy and select top 5 variables
importance_df_accuracy <-
importance_df[order(importance_df$MeanDecreaseAccuracy, decreasing = TRUE), ]
top5_accuracy <- importance_df_accuracy[1:5, ]

# Create a lollipop chart
# Order variables by MeanDecreaseAccuracy from highest to lowest
top5_accuracy$Variable <- factor(top5_accuracy$Variable, levels =
top5_accuracy$Variable[order(-top5_accuracy$MeanDecreaseAccuracy)])

# Create the lollipop chart
ggplot(top5_accuracy, aes(x = Variable, y = MeanDecreaseAccuracy)) +
  geom_point(color = "royalblue3", size = 5) +
  geom_segment(aes(x = Variable, xend = Variable, y = 0, yend =
MeanDecreaseAccuracy), color = "royalblue3") +
  labs(title = "Random Forest Modelindeki En Önemli 5 Değişken",
       x = "Değişkenler",
       y = "Ortalama Azalma Doğruluğu") +
  theme_minimal()
```



- The chart shows how the model changes over time. Sometimes lots of changes happen, other times not many. This means people are still working on the model, trying to make it better.It also shows the five most important things in the model: cd420, offtrt, treat, symptom, and time. These things have a big effect on what the

model predicts.In short, the chart tells us the model is getting better and more complicated as people keep working on it.

CONCLUSION

- In conclusion, the model developed to predict individuals infected with AIDS can be further improved to speed up the process in the healthcare sector, allowing patients to start treatment earlier. Using machine learning, our model has shown a performance with an 89% success rate and accurate predictions. This can enable faster and more accurate diagnoses in healthcare, significantly improving patient care. It can optimize the treatment processes for AIDS patients and enhance overall health outcomes.